

LeetCode 每日背诵清单

顺序按你当前文件名整理：No.1 -> No.2 -> No.3 -> No.4 -> No.5。

每道题结构：题目、源码注释解析、易错点、函数代码。

每日复盘

日期	背诵范围	是否能独立写出	卡住点	明天重点

No.1

1. 两数之和

- 文件：No.1_两数之和.cpp
- 源码注释解析：
 - 这题的关键是利用 `unordered_map` 查找平均 $O(1)$ 的特点，把暴力双循环降成一遍扫描。
 - 哈希表的 key 存数组里的值，value 存这个值对应的下标。
 - 遍历到 `nums[i]` 时，不是先存当前数，而是先查 `target - nums[i]` 是否已经出现。
 - 如果补数已经在哈希表里，说明当前数和之前某个数可以组成答案，直接返回两个下标。
 - 如果没有找到补数，再把当前数和下标存进去，给后面的元素使用。
- `unordered_map` 复习：
 - `mp[key] = value` 可以插入或修改。
 - `mp.find(key)` 查找 key，找不到时返回 `mp.end()`。
 - `mp.count(key)` 可以判断 key 是否存在。
 - 遍历哈希表可以用 `for (auto& item : mp)`，其中 `item.first` 是 key，`item.second` 是 value。
 - 删除元素可以用 `mp.erase(key)`。
- 易错点：先查再存，避免同一个元素被用两次。
- 复杂度：时间 $O(n)$ ，空间 $O(n)$ 。

```
vector<int> twosum(vector<int>& nums, int target) {
    unordered_map<int, int> mp;
    for (int i = 0; i < nums.size(); i++) {
        auto it = mp.find(target - nums[i]);
        if (it != mp.end()) {
            return {it->second, i};
        }
        mp[nums[i]] = i;
    }
    return {-1, -1};
}
```

49. 字母异位词分组

- 文件：No.1_字母异位词分组.cpp
- 源码注释解析：
 1. 虽然源码里没有长注释，但代码思路很固定：字母异位词只是字母顺序不同，排序后一定相同。
 2. 对每个字符串复制一份，排序后作为哈希表 key。
 3. 哈希表 value 是一个 `vector<string>`，用来保存所有属于同一组的原始字符串。
 4. 最后遍历哈希表，把每个 value 放进答案。
- 易错点：排序的是复制出来的 key，答案里要放原字符串。
- 复杂度：设字符串平均长度为 k，时间 $O(n * k \log k)$ ，空间 $O(nk)$ 。

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> mp;
    for (string s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        mp[key].push_back(s);
    }

    vector<vector<string>> ans;
    for (auto& item : mp) {
        ans.push_back(item.second);
    }
    return ans;
}
```

128. 最长连续序列

- 文件：No.1_最长连续序列.cpp
- 源码注释解析：
 1. 把所有数字放进 `unordered_set`，这样判断某个数字是否存在就是平均 $O(1)$ 。
 2. 如果一个数 x 的前一个数 $x - 1$ 存在，说明 x 不是连续序列的起点，直接跳过。
 3. 只有当 $x - 1$ 不存在时，才从 x 开始向后查 $x + 1, x + 2 \dots$ 。
 4. 每找到一个连续数字，当前长度加一。
 5. 每个连续段只会从最小值开始统计一次，所以整体仍然是 $O(n)$ 。
- 易错点：每个连续段起点的长度要重新从 1 开始。
- 复杂度：时间 $O(n)$ ，空间 $O(n)$ 。

```
int findLongest(vector<int>& nums) {
    unordered_set<int> st(nums.begin(), nums.end());
    int ans = 0;

    for (int x : st) {
        if (st.find(x - 1) != st.end()) continue;

        int cur = x;
        int len = 1;
        while (st.find(cur + 1) != st.end()) {
            cur++;
            len++;
        }
    }
    return ans;
}
```

```
    }  
    ans = max(ans, len);  
}  
  
return ans;  
}
```

No.2

2. 两数相加

- 文件：No.2_LiangShuXiangJia.cpp
- 源码注释解析：
 1. 两个链表表示两个非负整数，并且低位在前，所以可以直接从链表头开始逐位相加。
 2. 每一位的结果由三部分组成：l1 当前值、l2 当前值、上一位进位 carry。
 3. 新节点的值是 $\text{sum} \% 10$ 。
 4. 下一轮进位是 $\text{sum} / 10$ 。
 5. 两个链表长度可能不同，空链表位置按 0 处理。
 6. 循环结束后如果 carry 还有值，要再补一个节点。
- 易错点：两个链表长度可能不同，最后的进位不能漏。
- 复杂度：时间 $O(\max(m, n))$ ，空间 $O(\max(m, n))$ 。

```
struct ListNode {  
    int val;  
    ListNode* next;  
    ListNode() : val(0), next(nullptr) {}  
    ListNode(int x) : val(x), next(nullptr) {}  
    ListNode(int x, ListNode* next) : val(x), next(next) {}  
};  
  
ListNode* addSum(ListNode* l1, ListNode* l2) {  
    ListNode dummy(0);  
    ListNode* tail = &dummy;  
    int carry = 0;  
  
    while (l1 || l2 || carry) {  
        int n1 = l1 ? l1->val : 0;  
        int n2 = l2 ? l2->val : 0;  
        int sum = n1 + n2 + carry;  
  
        tail->next = new ListNode(sum % 10);  
        tail = tail->next;  
        carry = sum / 10;  
  
        if (l1) l1 = l1->next;  
        if (l2) l2 = l2->next;  
    }  
}
```

```
    return dummy.next;
}
```

15. 三数之和

- 文件: [No.2_三数之和.cpp](#)
- 源码注释解析:
 1. 先排序, 让数组有序, 后面才能使用双指针。
 2. 枚举第一个数 `nums[i]`, 剩下两个数在区间 `[i + 1, n - 1]` 里找。
 3. 左指针 `l` 指向较小数, 右指针 `r` 指向较大数。
 4. 如果三数之和等于 0, 记录答案, 并跳过左右两边的重复值。
 5. 如果和大于 0, 说明右边太大, `r--`。
 6. 如果和小于 0, 说明左边太小, `l++`。
 7. 固定的第一个数也要去重, 否则答案会重复。
- 易错点: 固定数去重要写 `continue`, 不能在 `if` 后面多写分号。
- 复杂度: 时间 $O(n^2)$, 空间 $O(1)$ 。

```
vector<vector<int>> ThreeSum(vector<int>& nums) {
    vector<vector<int>> ans;
    sort(nums.begin(), nums.end());

    for (int i = 0; i < nums.size(); i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int l = i + 1;
        int r = nums.size() - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum == 0) {
                ans.push_back({nums[i], nums[l], nums[r]});
                while (l < r && nums[l] == nums[l + 1]) l++;
                while (l < r && nums[r] == nums[r - 1]) r--;
                l++;
                r--;
            } else if (sum > 0) {
                r--;
            } else {
                l++;
            }
        }
    }

    return ans;
}
```

42. 接雨水

- 文件: [No.2_接雨水.cpp](#)

- 源码注释解析：
 1. 对某个位置来说，它能接的水由左右两边最高柱子的较小值决定。
 2. 用左指针 `l` 和右指针 `r` 从两端向中间走。
 3. `lm` 表示从左到当前位置遇到的最高柱子，`rm` 表示从右到当前位置遇到的最高柱子。
 4. 如果 `lm < rm`，说明左边短板已经确定，左侧位置最多只能接到 `lm`，所以结算左边。
 5. 否则右边短板已经确定，结算右边。
 6. 每次累加当前短板高度减去当前位置高度。
- 易错点：先更新 `lm / rm`，再累加当前位置水量。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
int JieShui(vector<int>& height) {
    int l = 0;
    int r = height.size() - 1;
    int lm = 0;
    int rm = 0;
    int ans = 0;

    while (l < r) {
        lm = max(lm, height[l]);
        rm = max(rm, height[r]);

        if (lm < rm) {
            ans += lm - height[l];
            l++;
        } else {
            ans += rm - height[r];
            r--;
        }
    }

    return ans;
}
```

11. 盛最多水的容器

- 文件：No.2_盛最多水的容器.cpp
- 源码注释解析：
 1. 容器面积等于 $\min(\text{height}[l], \text{height}[r]) * (r - l)$ 。
 2. 宽度会随着指针移动不断变小，所以想让面积变大，只能尝试提高短板。
 3. 因此每次移动较短的一边。
 4. 如果移动较高的一边，短板没有变高，宽度又变小，面积不可能更优。
- 易错点：不要移动高的一边，因为宽度变小且短板没变。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
int Maxarea(vector<int>& height) {
    int l = 0;
    int r = height.size() - 1;
    int ans = 0;
```

```

while (l < r) {
    int area = min(height[l], height[r]) * (r - l);
    ans = max(ans, area);

    if (height[l] < height[r]) {
        l++;
    } else {
        r--;
    }
}

return ans;
}

```

283. 移动零

- 文件：No.2_移动零.cpp
- 源码注释解析：
 1. 用慢指针 `slow` 表示下一个非零元素应该放的位置。
 2. 用快指针 `fast` 从左到右扫描所有元素。
 3. 当 `nums[fast] != 0` 时，把它交换到 `slow` 位置，然后 `slow++`。
 4. 这样所有非零元素会保持原来的相对顺序，所有 0 会自然被换到后面。
- 易错点：遇到非零才交换并移动慢指针。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```

void MoveZero(vector<int>& nums) {
    int slow = 0;
    for (int fast = 0; fast < nums.size(); fast++) {
        if (nums[fast] != 0) {
            swap(nums[slow], nums[fast]);
            slow++;
        }
    }
}

```

No.3

438. 找到字符串中所有字母异位词

- 文件：No.3_找到字符串中的所有字母异位词.cpp
- 源码注释解析：
 1. 使用固定长度滑动窗口，窗口大小等于 `p.size()`。
 2. 窗口在 `s` 上从左往右滑动，每次判断窗口内字符计数是否和 `p` 一致。
 3. 因为题目只包含小写英文字母，所以可以开两个长度为 26 的数组。
 4. `pcount[26]` 统计 `p` 中每个字母出现次数，这个数组固定不变。
 5. `scount[26]` 统计当前窗口内 `s` 的字母出现次数，窗口滑动时动态更新。
 6. 如果 `scount == pcount`，说明当前窗口是 `p` 的异位词，记录窗口左边界。

7. `vector<int>` 可以直接用 `==` 比较 26 个字母的计数是否完全相等。

8. 边界要注意最后一个窗口，循环结束后仍然要判断一次，或者把循环写成能覆盖最后窗口的形式。

- 易错点：最后一个窗口也要判断；`p.size() > s.size()` 直接返回空。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
vector<int> findAnagrams(string& s, string& p) {
    vector<int> ans;
    int n = s.size();
    int m = p.size();
    if (m > n) return ans;

    vector<int> scount(26, 0);
    vector<int> pcount(26, 0);

    for (int i = 0; i < m; i++) {
        scount[s[i] - 'a']++;
        pcount[p[i] - 'a']++;
    }

    for (int l = 0, r = m; r <= n; l++, r++) {
        if (scount == pcount) ans.push_back(l);
        if (r == n) break;
        scount[s[l] - 'a']--;
        scount[s[r] - 'a']++;
    }

    return ans;
}
```

3. 无重复字符的最长子串

- 文件：No.3_无重复字符的最长连续子串.cpp
- 源码注释解析：
 1. 窗口定义可以理解为 `[l, r]`，窗口内永远保存不含重复字符的连续子串。
 2. 用数组 `cnt[300]` 当作 ASCII 字符哈希表，记录窗口内每个字符出现次数。
 3. 右指针不断扩张窗口，把新字符加入窗口。
 4. 如果新加入的字符导致重复，就不断移动左指针，直到重复字符被移出，窗口重新合法。
 5. 整体逻辑一句话：右指针不断扩张窗口，遇到重复就收缩左指针，全程记录合法窗口最大长度。
- 易错点：更新答案时用当前合法窗口长度 `r - l + 1`。
- 复杂度：时间 $O(n)$ ，空间 $O(\text{字符集大小})$ 。

```
int ZiChuan(string& s) {
    int cnt[300] = {0};
    int l = 0;
    int ans = 0;

    for (int r = 0; r < s.size(); r++) {
```

```

        cnt[s[r]]++;
        while (cnt[s[r]] > 1) {
            cnt[s[l]]--;
            l++;
        }
        ans = max(ans, r - l + 1);
    }

    return ans;
}

```

No.4

560. 和为 K 的子数组

- 文件：No.4_和为K的子数组.cpp
- 源码注释解析：
 1. 题目数组里可能包含正数、负数和 0，所以不能使用普通的伸缩滑动窗口。
 2. 暴力双循环会超时，因此采用“前缀和 + 哈希表”的 $O(n)$ 解法。
 3. 定义 $pre[i]$ 为前 i 项和，并人为规定 $pre[0] = 0$ 。
 4. 子数组 $[l, r]$ 的和为 $pre[r + 1] - pre[l]$ 。
 5. 如果子数组和要等于 k ，就有 $pre[r + 1] - pre[l] = k$ 。
 6. 变形得到 $pre[l] = pre[r + 1] - k$ 。
 7. 遍历到当前位置时，当前前缀和记为 pre ，只需要查之前出现过多少次 $pre - k$ 。
 8. 哈希表记录的是“某个前缀和出现过几次”，出现几次就能新增几个符合条件的子数组。
 9. 初始化 $mp[0] = 1$ ，因为当前缀和刚好等于 k 时，从数组开头到当前位置也算一个答案。
- 易错点：必须初始化 $mp[0] = 1$ ，并且要先查答案，再记录当前前缀和。
- 复杂度：时间 $O(n)$ ，空间 $O(n)$ 。

```

int subarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> mp;
    int pre = 0;
    int ans = 0;
    mp[0] = 1;

    for (int x : nums) {
        pre += x;
        if (mp.find(pre - k) != mp.end()) {
            ans += mp[pre - k];
        }
        mp[pre]++;
    }

    return ans;
}

```

76. 最小覆盖子串

- 文件：No.4_最小覆盖字符串.cpp
- 源码注释解析：
 1. 先预处理 `t`，用 `need` 数组记录每个字符的需求数量。
 2. `kind` 记录 `t` 中不同字符的种类数。
 3. `win` 记录当前窗口中每个字符的数量。
 4. `match` 记录当前窗口中已经“数量满足要求”的字符种类数。
 5. 右指针 `r` 扩张窗口，每加入一个字符，就更新 `win`。
 6. 如果某个字符数量刚好达到需求，即 `win[c] == need[c]`，说明这个字符种类匹配齐了，`match++`。
 7. 当 `match == kind` 时，说明窗口已经合法，当前窗口覆盖了 `t`。
 8. 窗口合法后开始收缩左边界 `l`，每次收缩前更新最短子串答案。
 9. 如果移出的左侧字符原本是刚好满足需求的字符，移出后会破坏合法性，所以 `match--`。
 10. 最后如果没有找到合法窗口，返回空字符串；否则返回最短窗口。
- 易错点：只有字符数量刚好满足需求时，`match` 才变化；移出字符前要判断是否会破坏匹配。
- 复杂度：时间 $O(n)$ ，空间 $O(\text{字符集大小})$ 。

```
string minWindow(string& s, string& t) {
    int need[128] = {0};
    int win[128] = {0};
    int kind = 0;

    for (char c : t) {
        if (need[c] == 0) kind++;
        need[c]++;
    }

    int match = 0;
    int start = 0;
    int len = INT_MAX;
    int l = 0;

    for (int r = 0; r < s.size(); r++) {
        win[s[r]]++;
        if (win[s[r]] == need[s[r]]) match++;

        while (match == kind) {
            if (r - l + 1 < len) {
                len = r - l + 1;
                start = l;
            }

            if (win[s[l]] == need[s[l]]) match--;
            win[s[l]]--;
            l++;
        }
    }

    return len == INT_MAX ? "" : s.substr(start, len);
}
```

239. 滑动窗口最大值

- 文件：No.4_滑动窗口最大值.cpp
- 源码注释解析：
 1. 优化方案是使用单调递减双端队列 `deque`，整体时间复杂度 $O(n)$ 。
 2. 队列里只存数组下标，不直接存值。
 3. 队列维护规则：队列中下标对应的数值，从队头到队尾严格单调递减。
 4. 因此队头永远是当前窗口最大值的下标。
 5. 新元素入队前，把队尾所有小于等于当前值的下标全部弹出。
 6. 这些被弹出的元素不可能成为后续窗口最大值，因为它们更小，并且位置更靠左。
 7. 窗口右移后，要检查队头下标是否滑出窗口左边界，超出则弹出队头。
 8. 当窗口长度达到 k 后，每一步都把 `nums[deque.front()]` 加入答案。
- 易错点：队列里存下标；判断过期用 `que.front() <= i - k`。
- 复杂度：时间 $O(n)$ ，空间 $O(k)$ 。

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> que;
    vector<int> ans;

    for (int i = 0; i < nums.size(); i++) {
        while (!que.empty() && que.front() <= i - k) {
            que.pop_front();
        }

        while (!que.empty() && nums[i] >= nums[que.back()]) {
            que.pop_back();
        }

        que.push_back(i);

        if (i >= k - 1) {
            ans.push_back(nums[que.front()]);
        }
    }

    return ans;
}
```

No.5

56. 合并区间

- 文件：No.5_合并区间.cpp
- 源码注释解析：
 1. 第一步：先按照每个区间的左端点从小到大排序。
 2. 第二步：逐个遍历区间并合并。
 3. 使用一个新数组保存合并后的结果。
 4. 如果结果数组为空，直接把当前区间放进去。

5. 否则拿结果数组最后一个区间的右端点，和当前区间左端点比较。
 6. 如果结果最后区间的右端点小于当前左端点，说明没有重叠，直接新增当前区间。
 7. 否则说明两个区间重叠，需要合并。
 8. 合并方式是更新结果最后一个区间的右端点，取两个右端点里的最大值。
- 易错点：比较的是 `ans.back()[1]` 和当前左端点。
 - 复杂度：时间 $O(n \log n)$ ，空间 $O(n)$ 。

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    if (intervals.size() <= 1) return intervals;

    sort(intervals.begin(), intervals.end());
    vector<vector<int>> ans;

    for (auto& item : intervals) {
        int l = item[0];
        int r = item[1];

        if (ans.empty() || ans.back()[1] < l) {
            ans.push_back({l, r});
        } else {
            ans.back()[1] = max(ans.back()[1], r);
        }
    }

    return ans;
}
```

53. 最大子数组和

- 文件：No.5_最大子数组和.cpp
- 源码注释解析：
 1. 用两个变量。
 2. `cur` 表示“以当前数字结尾的最大连续和”。
 3. `ans` 表示全程记录的最大结果。
 4. 遍历每个数字时，如果前面的和加上当前数，不如只取当前数，说明前面的部分拖后腿，要丢掉前面，从当前数重新开始。
 5. 状态转移就是 `cur = max(cur + nums[i], nums[i])`。
 6. 每次更新完 `cur`，都用它更新全局最大值 `ans`。
- 易错点：初始化用 `nums[0]`，不要用固定哨兵值。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
int maxSubArray(vector<int>& nums) {
    int cur = nums[0];
    int ans = nums[0];

    for (int i = 1; i < nums.size(); i++) {
        cur = max(cur + nums[i], nums[i]);
        ans = max(ans, cur);
    }
}
```

```

    }

    return ans;
}

```

41. 缺失的第一个正数

- 文件：No.5_缺失的第一个正数.cpp
- 源码注释解析：
 1. 根据抽屉原理，长度为 n 的数组中，缺失的第一个正整数只可能在 $[1, n + 1]$ 范围内。
 2. 小于等于 0 的数，以及大于 n 的数，都不影响答案。
 3. 使用原地哈希：把范围 $[1, n]$ 内的数字 x 放到它应该在的位置，也就是下标 $x - 1$ 。
 4. 例如数字 1 应该放在下标 0，数字 2 应该放在下标 1。
 5. 遍历数组时，如果当前数在 $[1, n]$ 内，就尝试把它交换到正确位置。
 6. 如果目标位置已经是同一个数字，说明遇到了重复数字，要停止交换，避免死循环。
 7. 归位结束后再遍历数组，找到第一个满足 $\text{nums}[i] \neq i + 1$ 的位置，返回 $i + 1$ 。
 8. 如果所有位置都匹配，说明 $[1, n]$ 都存在，答案就是 $n + 1$ 。
 9. 边界重点：这里要用 `while`，不能只用 `if`，因为换过来的新数字可能还需要继续检查并归位。
- 易错点：交换必须用 `while`；遇到重复数字要停，防止死循环。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```

int firstMissingPositive(vector<int>& nums) {
    int n = nums.size();

    for (int i = 0; i < n; i++) {
        while (nums[i] >= 1 && nums[i] <= n) {
            int to = nums[i] - 1;
            if (nums[to] == nums[i]) break;
            swap(nums[to], nums[i]);
        }
    }

    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1) return i + 1;
    }

    return n + 1;
}

```

189. 轮转数组

- 文件：No.5_轮转数组.cpp
- 源码注释解析：
 1. 使用三步反转。
 2. 第一步：整体反转数组。
 3. 第二步：反转前 k 个元素。
 4. 第三步：反转剩下的后半部分。

5. 右轮转超过数组长度时会重复，所以要先做 $k \% \text{nums.size}()$ ，消除多余轮转次数。

- 易错点：如果数组可能为空，要先判断 $n == 0$ ，否则 $k \% n$ 会出错。
- 复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

```
void rotate(vector<int>& nums, int k) {
    int n = nums.size();
    if (n == 0) return;

    k %= n;
    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin() + k);
    reverse(nums.begin() + k, nums.end());
}
```

238. 除自身以外数组的乘积

- 文件：No.5_除了自身以外数组的乘积.cpp
- 源码注释解析：
 1. 题目要求不能使用除法，所以要把乘积拆开。
 2. 除 $\text{nums}[i]$ 以外的乘积，等于 i 左边所有数的乘积乘以 i 右边所有数的乘积。
 3. 可以用前缀乘积数组记录从开头到当前位置之前的乘积。
 4. 也可以用后缀乘积数组记录从末尾到当前位置之后的乘积。
 5. 最终 $\text{ans}[i] = \text{左侧全部乘积} * \text{右侧全部乘积}$ 。
 6. 更适合背诵的写法是只用答案数组：第一遍从左到右把左侧乘积写进 $\text{ans}[i]$ ，第二遍从右到左用变量 right 乘上右侧乘积。
 7. 这样不需要额外开前缀数组和后缀数组，空间更省。
- 易错点：推荐背“一遍左乘积，一遍右乘积”，不用除法。
- 复杂度：时间 $O(n)$ ，额外空间 $O(1)$ ，不算答案数组。

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, 1);

    int left = 1;
    for (int i = 0; i < n; i++) {
        ans[i] = left;
        left *= nums[i];
    }

    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        ans[i] *= right;
        right *= nums[i];
    }

    return ans;
}
```